# 3-D Object Representation

**Computer Graphics | Unit 3 | Classroom Notes**

Polygon surfaces • Quadric surfaces • Spline representation • Hermite, Bézier and B-spline curves • Bézier and B-spline surfaces • Polygon rendering methods • Colour models and colour applications

### How to use these notes

- **Colour code:** blue = definition, amber = important point, green = example, red = common mistake, purple = activity, teal = real-world link.
- **Prerequisites:** vectors (dot and cross product), matrices, polynomials, and the 2-D transformation and viewing notes.
- All numerical examples in these notes were checked by calculation. Where books use different notation (e.g. d or k for the B-spline order), follow your prescribed textbook in the exam.
- These notes do not assume any particular university syllabus or textbook. Check your syllabus for exact depth, especially for the colour-model and colour-application part.

| Topic map | What you will learn |
|---|---|
| 1–2. Polygon and quadric surfaces | How solid objects are stored as meshes and as equations |
| 3. Spline representation | Smooth curves from control points, continuity |
| 4–7. Hermite, Bézier, B-spline curves | Equations, matrices, properties, examples |
| 8. Bézier and B-spline surfaces | Extending curves to patches |
| 9. Polygon rendering | Flat, Gouraud and Phong shading |
| 10. Colour models and applications | RGB, CMY(K), YIQ, HSV/HLS and where each is used |

## 1 Introduction: How Do We Describe a 3-D Object?

A 3-D object must be stored in the computer in a form that can be **drawn, transformed, shaded and edited**. Different objects need different representations: a cube is easy as flat faces, a sphere as an equation, a car body as smooth curved patches.

### TWO BROAD FAMILIES (as commonly described in textbooks)

- **Boundary representations (B-reps):** describe the **surface** that separates the inside of the object from the outside. Examples: polygon surfaces, quadric surfaces, spline surfaces (this unit).
- **Space-partitioning representations:** divide the **space inside** the object into small solid pieces. Examples: octrees, BSP trees (not part of this topic list).

| Representation | Best for | Real-world example |
|---|---|---|
| Polygon mesh | Fast rendering, any shape (approximate) | Game characters, buildings in a 3-D map |
| Quadric surface | Simple curved shapes with exact equations | Planets, balls, tyres, pipes |
| Bézier / B-spline surface | Smooth, designable free-form shapes | Car bodies, aircraft wings, the Utah teapot |
| Splines (curves) | Smooth paths and outlines | Fonts, vector logos, animation paths, roller-coaster tracks |

# 2 Polygon Surfaces

**DEFINITIONS**

- **Polygon surface (polygon mesh):** an object surface described by a set of connected polygons (usually triangles or quadrilaterals).
- **Polygon table:** the stored data for a mesh: **vertex table** (coordinates), **edge table** (which two vertices form each edge) and **surface/polygon table** (which edges form each polygon).
- **Surface normal:** a vector perpendicular to a polygon's plane; it tells us which way the surface faces.

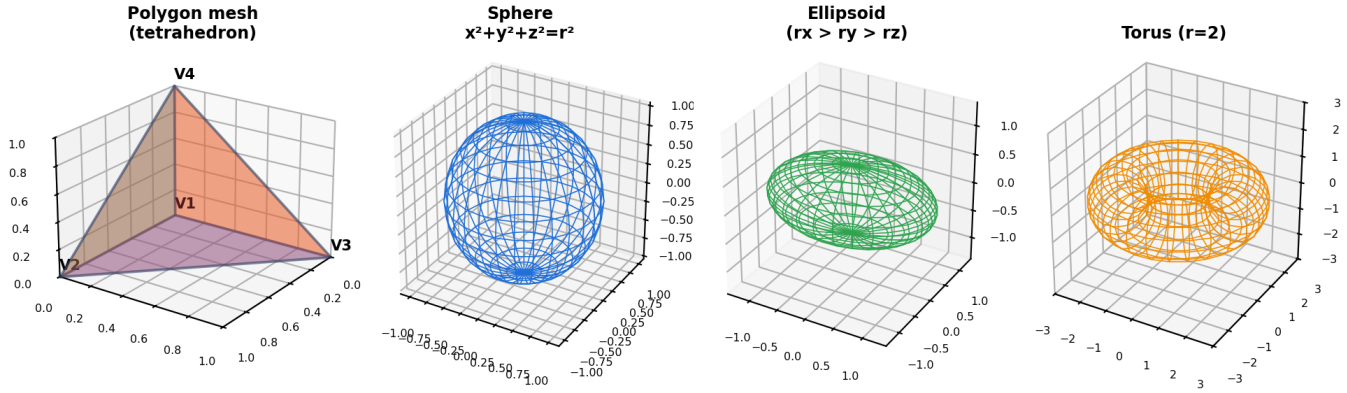



**Figure 1:** A polygon mesh (tetrahedron) and three common quadric surfaces: sphere, ellipsoid and torus.

## Example: tetrahedron with V1(0,0,0), V2(1,0,0), V3(0,1,0), V4(0,0,1)

| Vertex | Coordinates |
|---|---|
| V1 | (0,0,0) |
| V2 | (1,0,0) |
| V3 | (0,1,0) |
| V4 | (0,0,1) |

| Edge | Vertices |
|---|---|
| E1 | V1, V2 |
| E2 | V2, V3 |
| E3 | V3, V1 |
| E4 | V1, V4 |
| E5 | V2, V4 |
| E6 | V3, V4 |

| Surface | Edges |
|---|---|
| S1 (base) | E1, E2, E3 |
| S2 | E1, E5, E4 |
| S3 | E3, E6, E4 |
| S4 | E2, E6, E5 |

**WHY THREE TABLES?**

- Data is stored **once**: a shared vertex or edge is not repeated, so memory is saved.
- **Consistency checks:** every edge should belong to at least two polygons (exactly two in a closed solid), every vertex should be used by at least one edge, and every polygon should be closed. In the example, each edge E1…E6 appears in exactly two surfaces.

## Plane equation and normal

```
Ax + By + Cz + D = 0            normal vector  N = (A, B, C)

Given three non-collinear vertices P1, P2, P3 of the polygon (listed counter-clockwise
when viewed from the OUTSIDE of the object):
      N = (P2 - P1) x (P3 - P1)            (cross product)
      D = - N . P1
```

**INSIDE / OUTSIDE TEST**

With vertices ordered as above, so that N points **outward**: for a point (x, y, z), if **Ax + By + Cz + D < 0** the point is **inside** (behind the plane); if **> 0** it is **outside**; if = 0 it is on the plane.

**Example 1: Plane of face S4**

P1 = V2(1,0,0), P2 = V3(0,1,0), P3 = V4(0,0,1).
P2 − P1 = (−1, 1, 0), P3 − P1 = (−1, 0, 1). Cross product = (1·1 − 0·0, 0·(−1) − (−1)·1, (−1)·0 − 1·(−1)) = **(1, 1, 1)**.
D = −(1·1 + 0 + 0) = −1. **Plane: x + y + z − 1 = 0.**
Test the origin (0,0,0): 0 − 1 = −1 < 0, **inside**. Test (1,1,1): 3 − 1 = 2 > 0, **outside**.

**REAL-WORLD**

Game and film models are meshes of thousands to millions of triangles. Graphics hardware is built to draw triangles very fast. 3-D printing (STL files) also stores triangle meshes.

**IMPORTANT**

**Triangles are preferred** because three points always lie in one plane. A polygon with four or more vertices may be non-planar, which causes problems. A quadrilateral can be split into two triangles.

| Mesh type | Description | Use |
|---|---|---|
| Triangle strip | Series of triangles, each sharing an edge with the previous one; n triangles need n + 2 vertices | Saves memory and time |
| Quadrilateral mesh | n × m array of vertices gives (n−1)(m−1) quadrilaterals | Regular grids, terrain, surfaces |

## 3 Quadric Surfaces

**DEFINITION**

**Quadric surfaces** are surfaces described by **second-degree equations** (quadratics) in x, y and z. Common examples: sphere, ellipsoid, paraboloid, hyperboloid. Textbooks usually also list the **torus** in this group, although its full equation has a higher degree.

| Surface | Cartesian equation | Parametric form (angles in radians) |
|---|---|---|
| Sphere (radius r, centre origin) | $x^2 + y^2 + z^2 = r^2$ | $x = r\cos\varphi\cos\theta$, $y = r\cos\varphi\sin\theta$, $z = r\sin\varphi$<br>$-\pi/2 \le \varphi \le \pi/2$ (latitude), $0 \le \theta \le 2\pi$ (longitude) |
| Ellipsoid (radii rx, ry, rz) | $(x/rx)^2 + (y/ry)^2 + (z/rz)^2 = 1$ | $x = rx\cos\varphi\cos\theta$, $y = ry\cos\varphi\sin\theta$, $z = rz\sin\varphi$<br>same ranges of φ and θ |
| Torus (r = distance from centre of tube to axis, in units of the tube radius) | $[\, r - \sqrt{((x/rx)^2 + (y/ry)^2)}\,]^2 + (z/rz)^2 = 1$ | $x = rx\,(r + \cos\varphi)\cos\theta$, $y = ry\,(r + \cos\varphi)\sin\theta$, $z = rz\sin\varphi$<br>$-\pi \le \varphi \le \pi$, $0 \le \theta \le 2\pi$ |

**Example 2: Point on a sphere**

Sphere with r = 2. Find the point for φ = 30°, θ = 60°.
x = 2·cos30°·cos60° = 2(0.866)(0.5) = **0.866**; y = 2·cos30°·sin60° = 2(0.866)(0.866) = **1.5**; z = 2·sin30° = **1**.
Check: $x^2 + y^2 + z^2 = 0.75 + 2.25 + 1 = 4 = r^2$. ✓

| Aspect | Polygon mesh | Quadric surface |
|---|---|---|
| Accuracy | Approximate (flat facets) | Exact shape from an equation |
| Storage | Many vertices for smooth look | A few parameters (radii) |
| Generality | Any shape | Only specific shapes |
| Rendering | Direct and fast | Often converted to a mesh using θ, φ steps |

**REAL-WORLD**

Planets and balls (sphere), the Earth (slightly flattened ellipsoid), a doughnut or tyre (torus). CAD systems keep exact quadrics for cylinders, cones and spheres.

# 4 Spline Representation

**DEFINITIONS**

- **Spline (original meaning):** a flexible strip used by draftsmen to draw a smooth curve through a set of points.
- **Spline curve (in graphics):** a smooth curve made of **polynomial pieces** joined with specified **continuity** at the joints. A spline surface is built from two sets of spline curves.
- **Control points:** the points that define the shape. The polyline joining them is the **control polygon (control graph)**.
- **Interpolation spline:** the curve **passes through** the control points.
- **Approximation spline:** the curve follows the control points but **need not pass through** them.
- **Convex hull:** the smallest convex region containing all the control points (like a rubber band around them).

| | Interpolation | Approximation |
|---|---|---|
| Passes through control points? | Yes | Not necessarily |
| Typical use | Digitizing / tracing known points, animation keyframes | Designing shapes (CAD, fonts) |
| Examples here | Hermite, natural cubic spline | Bézier (ends only), B-spline |

## Continuity conditions at the join of two curve sections

| Parametric | Meaning | Geometric | Meaning |
|---|---|---|---|
| $C^0$ | The sections meet (same end point) | $G^0$ | Same as $C^0$ |
| $C^1$ | First derivatives (tangent vectors) are **equal** | $G^1$ | Tangent **directions** are the same; lengths may differ |
| $C^2$ | First and second derivatives are equal | $G^2$ | Curvature is equal |

**IMPORTANT**

$C^1$ continuity implies $G^1$, but not the other way round. **Cubic polynomials** are the most popular choice: they give enough freedom for $C^2$ smoothness without the wobbles of high-degree curves.

**HOW A SPLINE SECTION IS SPECIFIED**

- A set of **boundary conditions** (end points, end tangents, ...), or
- A **basis (characterizing) matrix** M such that $P(u) = [u^3\ u^2\ u\ 1] \cdot M \cdot [\text{geometry}]$, or
- A set of **blending functions** (basis functions) that tell how much each control point influences the curve at parameter u.

**REAL-WORLD**

Fonts (TrueType uses quadratic curves; PostScript and OpenType-CFF use cubic curves), the Pen tool in vector editors, CSS `cubic-bezier()` animation easing, smooth camera paths in games.

# 5 Hermite Curve

**DEFINITION**

A **Hermite curve** is a **cubic** polynomial curve section defined by its **two end points** P0 and P1 and the **tangent vectors** T0 and T1 at those end points. It is an **interpolation** spline.

$$P(u) = (2u^3 - 3u^2 + 1)\,P0 + (-2u^3 + 3u^2)\,P1 + (u^3 - 2u^2 + u)\,T0 + (u^3 - u^2)\,T1$$
$$0 \le u \le 1, \quad P(0) = P0,\ P(1) = P1,\ P'(0) = T0,\ P'(1) = T1$$

| $P(u) = [u^3\ u^2\ u\ 1] \cdot M_H \cdot G_H$ | | | | Geometry vector $G_H$ |
|---|---|---|---|---|
| 2 | −2 | 1 | 1 | P0 |
| −3 | 3 | −2 | −1 | P1 |
| 0 | 0 | 1 | 0 | T0 |
| 1 | 0 | 0 | 0 | T1 |

**Derivation idea:** write $P(u) = a\,u^3 + b\,u^2 + c\,u + d$ (four unknown vectors). Apply the four conditions $P(0) = P0$, $P(1) = P1$, $P'(0) = T0$, $P'(1) = T1$ and solve for a, b, c, d. The result gives the matrix $M_H$ above; its rows are the coefficients of $u^3$, $u^2$, u, 1.

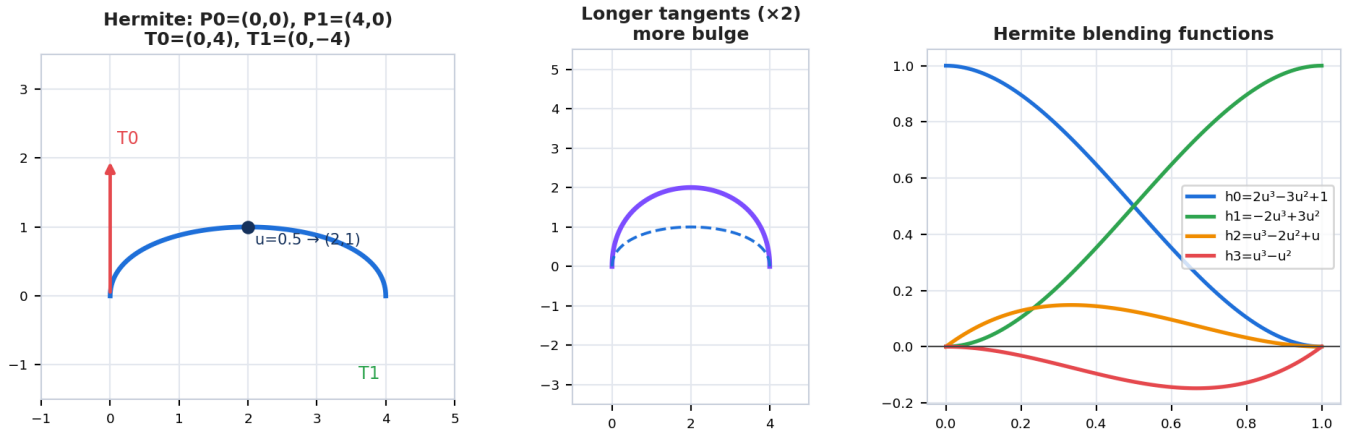



**Figure 2:** Hermite curve and the effect of tangent length. Right: the four Hermite blending functions.

**Example 3: Hermite point**

P0 = (0,0), P1 = (4,0), T0 = (0,4), T1 = (0,−4), u = 0.5.
Blending values at u = 0.5: h0 = 0.5, h1 = 0.5, h2 = 0.125, h3 = −0.125.
P(0.5) = 0.5(0,0) + 0.5(4,0) + 0.125(0,4) − 0.125(0,−4) = (2, 0.5 + 0.5) = **(2, 1)**.

| Property | Hermite curve |
|---|---|
| Passes through end points | Yes (interpolation) |
| Shape control | By the tangent vectors: larger tangent length gives more bulge |
| Continuity between segments | $C^1$ if the end tangent of one segment equals the start tangent of the next |
| Weakness | Tangent vectors are not intuitive for designers to specify |

**HERMITE ↔ BÉZIER**

A cubic Bézier curve with control points p0…p3 is the same curve as a Hermite curve with **P0 = p0, P1 = p3, T0 = 3(p1 − p0), T1 = 3(p3 − p2)**.

**REAL-WORLD**

Keyframe animation: a character's path passes through given positions (end points) with a chosen velocity (tangent) at each key.

# 6 Bézier Curve

**DEFINITION**

A **Bézier curve** of degree n is defined by n + 1 control points p0 ... pn. It is an **approximation** spline: it starts at p0, ends at pn, and is pulled towards the points in between.

```
P(u) = SUM for k = 0..n of  p_k · BEZ(k,n)(u),      0 <= u <= 1
BEZ(k,n)(u) = C(n,k) · u^k · (1-u)^(n-k),      C(n,k) = n! / (k! (n-k)!)
```

## Cubic Bézier curve (n = 3, four control points)

```
P(u) = (1-u)³ p0 + 3u(1-u)² p1 + 3u²(1-u) p2 + u³ p3
```

$$P(u) = [u^3\ u^2\ u\ 1] \cdot M_{Bez} \cdot [p0\ p1\ p2\ p3]^T$$

$$\begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

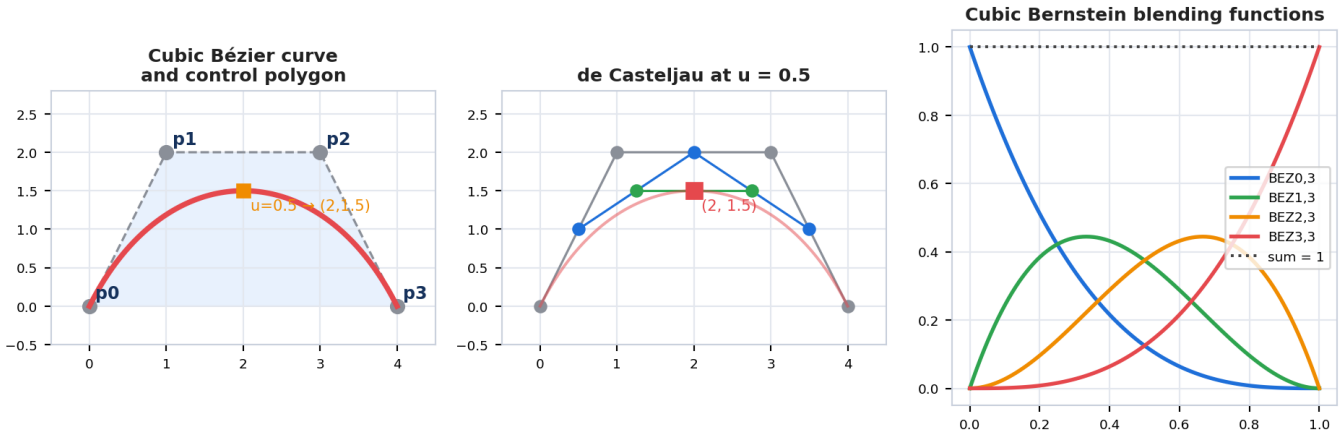



**Figure 3:** A cubic Bézier curve with its control polygon, the de Casteljau construction at u = 0.5, and the Bernstein blending functions (they always add up to 1).

**Example 4: Cubic Bézier points**

Control points p0 = (0,0), p1 = (1,2), p2 = (3,2), p3 = (4,0). At u = 0.5 the weights are 1/8, 3/8, 3/8, 1/8. x = 0 + 3/8 + 9/8 + 4/8 = 2, y = 0 + 6/8 + 6/8 + 0 = 1.5. **P(0.5) = (2, 1.5).**
At u = 0.25 the weights are 0.4219, 0.4219, 0.1406, 0.0156, giving **P(0.25) = (0.906, 1.125)**. Also P(0) = p0 = (0,0) and P(1) = p3 = (4,0).

**Example 5: Quadratic Bézier (n = 2)**

p0 = (0,0), p1 = (2,4), p2 = (4,0). BEZ weights at u = 0.5: 0.25, 0.5, 0.25. **P(0.5) = (2, 2).**

**de Casteljau construction (a simple way to compute a point)**

To find P(u): take the point that divides each control-polygon segment in the ratio u : (1 − u). This gives one point fewer. Repeat on the new points until only one point remains. That point is P(u). For u = 0.5 above: midpoints (0.5,1), (2,2), (3.5,1) → (1.25,1.5), (2.75,1.5) → **(2, 1.5)**.

## Properties of Bézier curves

| Property | Statement |
|---|---|
| End points | Curve passes through the first and last control points: P(0) = p0, P(1) = pn |
| End tangents | P′(0) = n(p1 − p0) and P′(1) = n(pn − p(n−1)): tangent along the first and last control-polygon segments |
| Convex hull | The whole curve lies inside the convex hull of its control points |
| Degree | Degree = number of control points − 1 (more points mean higher degree) |
| Blending functions | Non-negative and sum to 1 for every u; symmetric |
| Global control | Moving **one** control point changes the **whole** curve |
| Affine invariance | Transform the control points (rotate, scale, translate) and redraw: same as transforming the curve |

## Joining two cubic Bézier segments (p0…p3 and q0…q3)

- **$C^0$:** q0 = p3.
- **$G^1$:** in addition, p2, p3 = q0 and q1 lie on one straight line.
- **$C^1$:** in addition, q1 − q0 = p3 − p2, that is q1 = 2·p3 − p2.

**IMPORTANT**

High-degree Bézier curves are costly and can wiggle, so designers join several **cubic** segments instead. The lack of local control is the main reason B-splines were introduced.

# 7 B-Spline Curve

**DEFINITION**

A **B-spline curve** is a piecewise polynomial curve built from n + 1 control points p0 … pn and **B-spline basis functions** $B_{k,d}(u)$ of order d. Unlike a Bézier curve, its **degree (d − 1) does not depend on the number of control points**, and each control point affects only a **part** of the curve.

```
P(u) = SUM for k = 0..n of  p_k · B(k,d)(u),       u_min <= u <= u_max,      2 <= d <= n+1

Cox-de Boor recursion:
  B(k,1)(u) = 1  if  u_k <= u < u_(k+1),   else 0
  B(k,d)(u) = (u - u_k)/(u_(k+d-1) - u_k) · B(k,d-1)(u)  +  (u_(k+d) - u)/(u_(k+d) - u_(k+1)) · B
(k+1,d-1)(u)

Knot vector: n + d + 1 values  u_0 <= u_1 <= ... <= u_(n+d)      (a term 0/0 is taken as 0)
```

**TERMS**

- **Order d** and **degree d − 1**. Cubic B-spline: d = 4.
- **Knots:** parameter values where the polynomial pieces join. The list of them is the **knot vector**.
- **Uniform** knot vector: equal spacing, e.g. {0, 1, 2, 3, …}. Curve does **not** touch the end control points.
- **Open uniform** knot vector: first d and last d knots are equal, interior spacing uniform, e.g. for n = 5, d = 4: {0,0,0,0,1,2,3,3,3,3}. Curve starts at p0 and ends at pn.
- **Non-uniform** knot vector: unequal spacing or repeated interior knots. Gives more control (e.g. sharper corners).

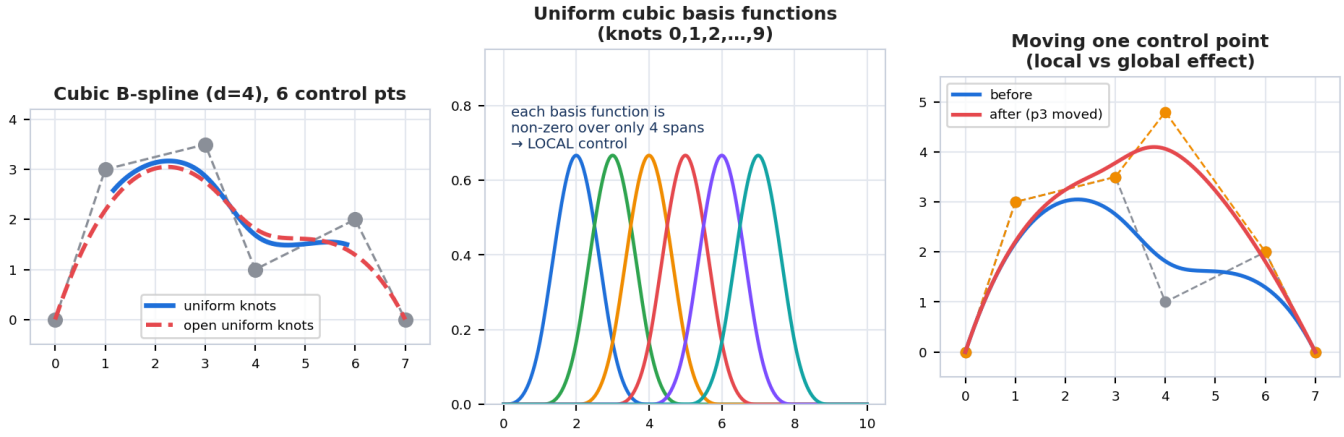



**Figure 4:** Left: a cubic B-spline with uniform and open uniform knots. Middle: uniform cubic basis functions (each is non-zero over only four spans). Right: moving p3 changes only a part of the curve.

## Uniform cubic B-spline segment

Each segment uses **four consecutive control points**. For one segment with points p0, p1, p2, p3 and $0 \le u \le 1$:

$$P(u) = [u^3\ u^2\ u\ 1] \cdot (1/6) \cdot M \cdot [p0\ p1\ p2\ p3]^T$$

$$\begin{vmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 0 & 3 & 0 \\ 1 & 4 & 1 & 0 \end{vmatrix}$$

**Example 6: Uniform cubic B-spline segment**

Control points (0,0), (1,2), (3,2), (4,0).
u = 0: P = (p0 + 4p1 + p2)/6 = ((0+4+3)/6, (0+8+2)/6) = **(1.167, 1.667)**.
u = 1: P = (p1 + 4p2 + p3)/6 = ((1+12+4)/6, (2+8+0)/6) = **(2.833, 1.667)**.
u = 0.5: weights 1/48, 0.4792, 0.4792, 1/48 give **(2, 1.917)**.
Compare with the Bézier curve on the same points: P(0.5) = (2, 1.5) and it touches (0,0) and (4,0). This B-spline segment **does not touch** the end control points and stays closer to the middle points.

## Properties of B-spline curves

| Property | Statement |
|---|---|
| Local control | Each control point influences only d knot spans; moving it changes only that part of the curve |
| Continuity | With distinct (simple) knots, the curve has $C^{d-2}$ continuity (cubic: $C^2$). A repeated knot lowers it |
| Convex hull | Each curve piece lies within the convex hull of the d control points that affect it |
| Degree control | Degree is chosen independently of the number of control points (need n + 1 ≥ d) |
| End points | Not interpolated by default; open uniform knots make the curve start at p0 and end at pn |
| Special case | With d = n + 1 and an open uniform knot vector, the B-spline curve is the **Bézier curve** |
| Partition of unity | Basis functions are non-negative and sum to 1 inside the valid range |

**KNOT COUNT CHECK**

Number of knots = n + d + 1. For 6 control points (n = 5) and a cubic (d = 4): 5 + 4 + 1 = **10 knots**. The valid curve range is $u_{d-1} \le u \le u_{n+1}$ for a plain uniform knot vector.

**REAL-WORLD**

CAD/CAM and animation software (NURBS, which are rational B-splines, are widely used in these tools); smooth camera paths; road and rail design where continuous curvature ($C^2$) matters.

## 8 Comparison: Hermite, Bézier and B-Spline Curves

| Feature | Hermite | Bézier | B-spline |
|---|---|---|---|
| Defined by | 2 end points + 2 tangents | n + 1 control points | n + 1 control points + knot vector + order d |
| Type | Interpolation | Approximation (touches ends only) | Approximation |
| Degree | 3 (cubic) | n (number of points − 1) | d − 1, chosen by the user |
| Control | Per segment (tangents) | Global: one point affects the whole curve | **Local**: one point affects d spans |
| Passes through end points | Yes | Yes | Only with open knot vector |
| Continuity when joining | $C^1$ if tangents match | $C^0$ / $C^1$ need conditions | $C^{d-2}$ automatically (simple knots) |
| Convex hull property | No (not defined by control points) | Yes | Yes (local hulls) |
| Ease of design | Tangents are hard to guess | Easy and intuitive | Very flexible, a little more complex |
| Typical use | Keyframe paths, interpolation | Fonts, vector graphics, simple shapes | CAD, complex smooth shapes and surfaces |

## 9 Bézier and B-Spline Surfaces

**TENSOR-PRODUCT IDEA**

A spline **surface** is built by using **two parameters u and v**: a grid (net) of control points is blended in the u direction by one set of curve functions and in the v direction by another. In effect, a surface is a curve whose control points are themselves moving along curves.

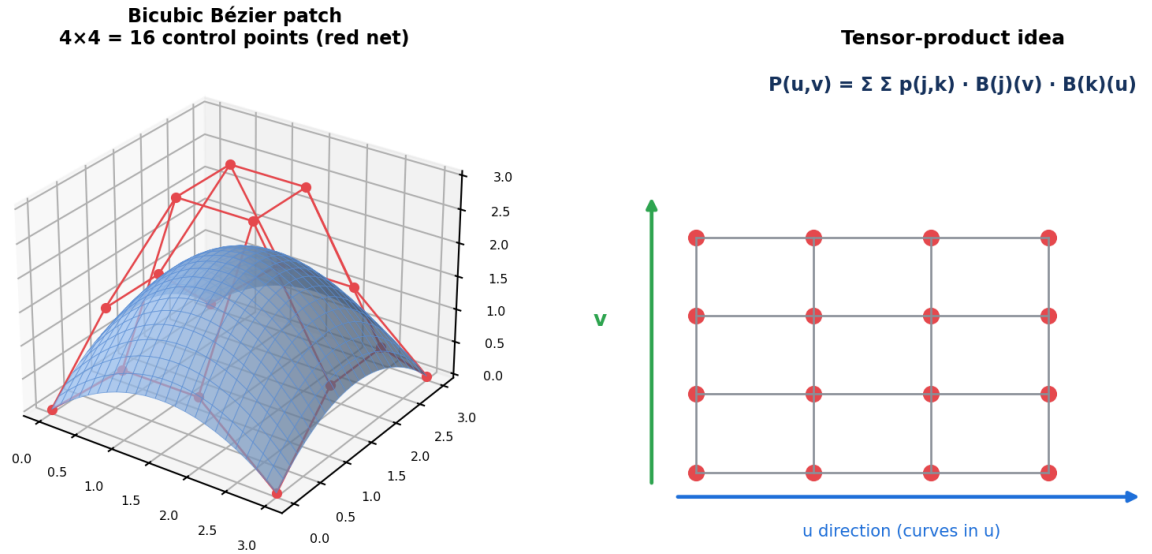



**Figure 5:** A bicubic Bézier patch. The red 4 × 4 net is the control grid; the blue surface follows it smoothly.

## Bézier surface

```
P(u,v) = SUM (j=0..m) SUM (k=0..n)  p(j,k) · BEZ(j,m)(v) · BEZ(k,n)(u),       0 <= u, v <= 1
Control net has (m+1) x (n+1) points.  Bicubic patch: m = n = 3  ->  16 control points
```

- The **four corner points** of the control net lie on the surface.
- The four **boundary curves** are Bézier curves defined by the boundary rows and columns of control points.
- The surface lies within the **convex hull** of the control net.
- Moving one control point changes the **whole** patch (global control).
- Joining two patches: $C^0$ if they share the same boundary control points; $C^1$ (smooth) if control points on both sides of the boundary are collinear with equal spacing (the surface analogue of curve $C^1$ continuity).

## B-spline surface

```
P(u,v) = SUM (k1=0..n1) SUM (k2=0..n2)  p(k1,k2) · B(k1,d1)(u) · B(k2,d2)(v)
Two knot vectors (one for u, one for v); orders d1 and d2 may be different
```

| Feature | Bézier surface | B-spline surface |
|---|---|---|
| Control net | (m+1) × (n+1) points; degrees are m and n | (n1+1) × (n2+1) points; degrees d1−1 and d2−1 chosen separately |
| Control | Global | **Local**: a control point changes only a patch region |
| Corner points on surface | Always | Only with open knot vectors |
| Large surfaces | Many Bézier patches must be joined by hand with continuity conditions | One B-spline surface gives smooth continuity automatically |
| Typical use | Single patches, e.g. teapot parts, small shapes | Car/aircraft bodies, character models, CAD |

**REAL-WORLD**

The famous Utah teapot is modelled with bicubic Bézier patches. Car bodies and ship hulls are designed with B-spline/NURBS surfaces.

# 10 Polygon Rendering Methods

**DEFINITION**

**Polygon rendering** means finding the **colour/intensity of every pixel** of a polygon-mesh surface. The **illumination model** gives the intensity at a point (from light, surface properties, directions); a **shading method** decides at which points the model is used and how the rest are filled in.

**A basic illumination model (Phong reflection model, for reference)**

$\mathbf{I = k_a I_a + k_d I_l (N \cdot L) + k_s I_l (R \cdot V)^{n_s}}$

Ambient + diffuse + specular. N = unit normal, L = unit vector to the light, V = unit vector to the viewer, R = reflection direction. Example: $k_d = 0.5$, $I_l = 1$, N = (0,0,1), L = (0, 0.6, 0.8): diffuse = 0.5 × 0.8 = **0.4**.

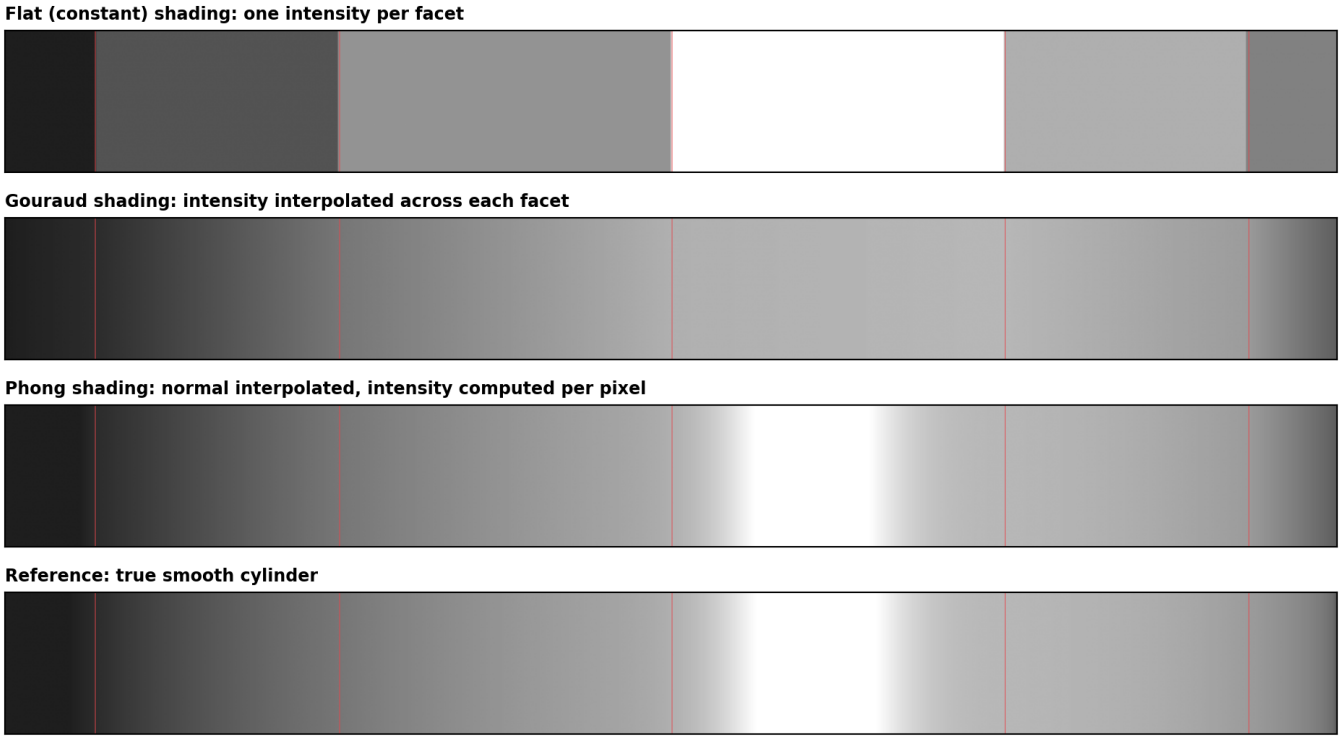



**Figure 6:** A cylinder approximated by 6 flat facets, shaded with the three methods. Flat shading shows bands; Gouraud is smooth but loses the bright highlight; Phong shows a good highlight.

## 10.1 Constant (flat) shading

- One normal and **one intensity per polygon** (the illumination model is evaluated once).
- Fastest. Good when the object really is flat-faced (cube) and the light and viewer are far away.
- For curved objects approximated by polygons, the facets are clearly visible; **Mach bands** (brighter and darker edges seen by the eye) exaggerate the look.

## 10.2 Gouraud shading (intensity interpolation)

1. **Vertex normal:** average the normals of all polygons that share the vertex: $N_v = \Sigma N_k / |\Sigma N_k|$.
2. Apply the illumination model at each **vertex** to get its intensity.
3. Interpolate the intensity **along polygon edges** down to each scan line.
4. Interpolate **along the scan line** between the two edge intensities.

```
Edge (vertices 1 and 2) at scan line y_s:   I_a = (y_s - y_2)/(y_1 - y_2) · I_1  +  (y_1 - y_s)/(y_1 - y_2) · I_2
Along the scan line, from x_a to x_b:     I_p = (x_b - x_p)/(x_b - x_a) · I_a  +  (x_p - x_a)/(x_b - x_a) · I_b
```

**Example 7: Gouraud interpolation**

Vertex 1: y = 10, I = 0.2; vertex 2: y = 2, I = 0.8. At scan line y = 6: I = (6−2)/(10−2)·0.2 + (10−6)/(10−2)·0.8 = 0.1 + 0.4 = **0.5**.
On a scan line with $I_a = 0.4$ at $x_a = 2$ and $I_b = 0.8$ at $x_b = 10$, at x = 4: I = (10−4)/8·0.4 + (4−2)/8·0.8 = 0.3 + 0.2 = **0.5**.
Vertex normal example: faces with normals (0,0,1), (0,1,0), (1,0,0) share a vertex. Sum = (1,1,1), length √3, so $N_v$ = **(0.577, 0.577, 0.577)**.

## 10.3 Phong shading (normal-vector interpolation)

1. Compute vertex normals (averaging, as in Gouraud).
2. **Interpolate the normal vector** along edges and then along scan lines.
3. At **every pixel**, normalize the interpolated normal and apply the illumination model.

**DON'T CONFUSE**

**Phong illumination (reflection) model** = the equation for intensity. **Phong shading** = the method that interpolates normals. Gouraud shading can also use the Phong reflection model at the vertices.

| Feature | Flat | Gouraud | Phong |
|---|---|---|---|
| Interpolates | Nothing | Intensity | Normal vector |
| Illumination model evaluated | Once per polygon | Once per vertex | Once per pixel |
| Speed | Fastest | Medium | Slowest |
| Smoothness | Faceted look | Smooth | Smoothest |
| Specular highlights | Poor | Can be missed or distorted if inside a polygon | Accurate, even inside a polygon |
| Mach bands | Strong | Reduced, can remain | Much reduced |
| Typical use | Previews, flat-faced objects | Real-time graphics in early/low-end systems | High-quality shading |

**REAL-WORLD**

Modern GPUs run small programs (fragment/pixel shaders) per pixel, so per-pixel normal-based shading (Phong-style) is now common in games.

# 11 Colour Models and Colour Applications

## 11.1 Basic light and colour terms

**DEFINITIONS**

- **Visible light:** electromagnetic waves of about 400–700 nm wavelength.
- **Hue (dominant wavelength):** the "colour name" we see (red, green, ...).
- **Saturation (purity):** how far a colour is from grey; pure spectral colours are fully saturated.
- **Brightness / luminance:** how much light energy is perceived.
- **Primary colours:** a set of colours that can be mixed to make other colours. **Complementary colours** mix to give white (in additive mixing).
- **Colour gamut:** the range of colours a device (monitor, printer) can reproduce.
- **Colour model:** a way of specifying colours numerically, usually with three values.

**CIE chromaticity (for awareness)**

The CIE system defines standard primaries X, Y, Z. Normalized values $x = X/(X+Y+Z)$, $y = Y/(X+Y+Z)$, $z = 1 - x - y$ are plotted in the **chromaticity diagram**, which is used to compare gamuts and describe colour mixing independent of brightness.

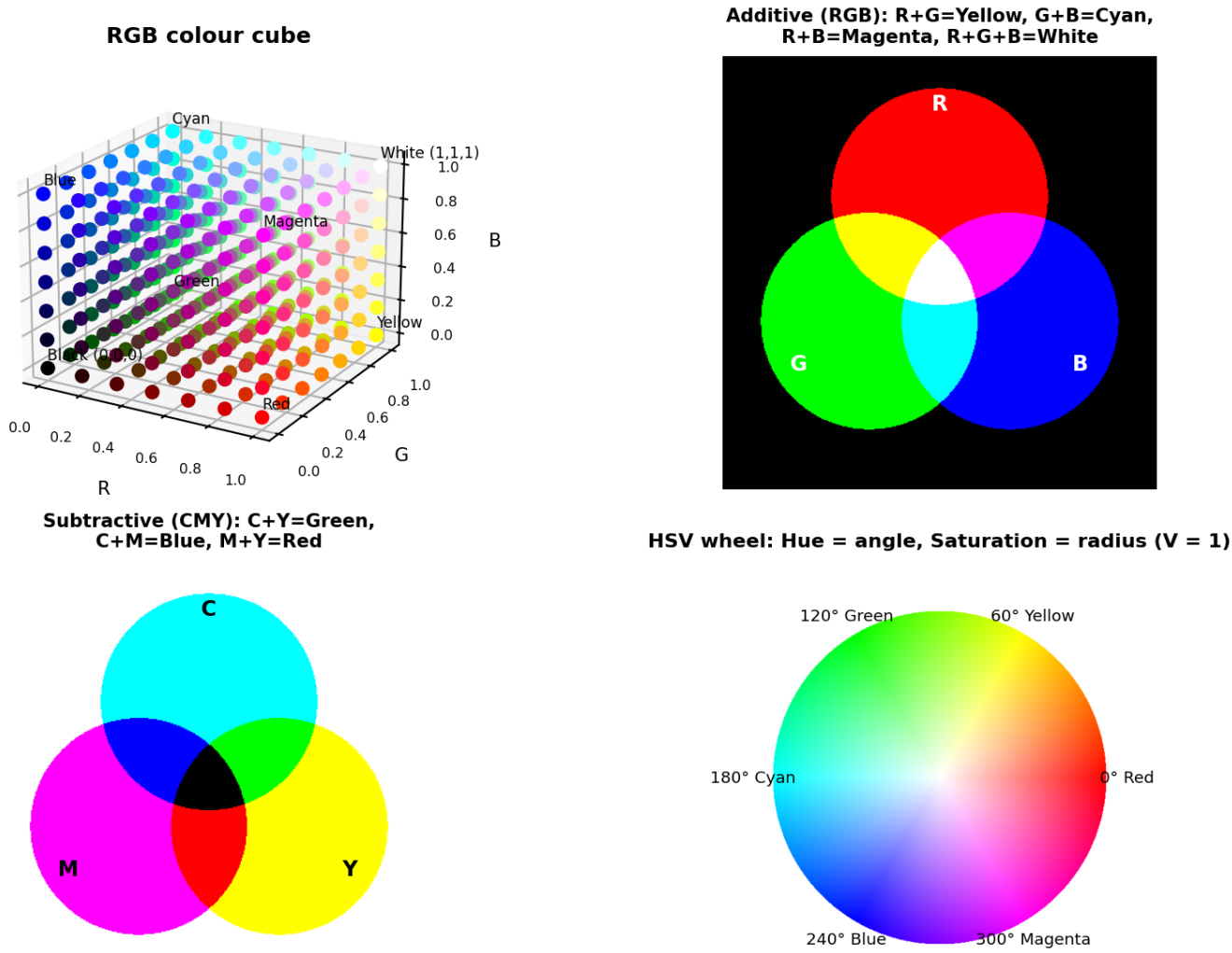



**Figure 7:** RGB cube, additive mixing, subtractive mixing and the HSV colour wheel.

## 11.2 The main colour models

| Model | Components | Idea | Main use |
|---|---|---|---|
| **RGB** | Red, Green, Blue, each 0–1 (or 0–255) | **Additive**: add light. (0,0,0) = black, (1,1,1) = white. Unit cube; grey on the diagonal. 24-bit colour: $256^3$ = 16,777,216 colours | Monitors, cameras, phones, web |
| **CMY / CMYK** | Cyan, Magenta, Yellow (+ Black K) | **Subtractive**: inks remove light. C = 1−R, M = 1−G, Y = 1−B. K added because real inks do not make a perfect black | Printing |
| **YIQ** | Y (luminance), I and Q (chrominance) | Separates brightness from colour. Y = 0.299R + 0.587G + 0.114B, so black-and-white sets can use Y only | NTSC TV, video, compression |
| **HSV** | Hue 0–360°, Saturation 0–1, Value 0–1 | Hexcone model close to how artists mix colours: pick a hue, then tint and shade | Colour pickers, image editing |
| **HLS** | Hue, Lightness, Saturation | Double-hexcone; L = 0 is black, L = 1 is white, pure colours at L = 0.5 | Colour pickers, CSS (hsl) |

## Conversions and formulas

```
RGB -> CMY :   C = 1 - R,   M = 1 - G,   Y = 1 - B          (CMY -> RGB the same way)
CMYK (simple form):  K = min(C,M,Y);   C' = (C-K)/(1-K),  M' = (M-K)/(1-K),  Y' = (Y-K)/(1-K)
RGB -> YIQ :   Y =  0.299 R + 0.587 G + 0.114 B
               I =  0.596 R - 0.275 G - 0.321 B
               Q =  0.212 R - 0.523 G + 0.311 B           (coefficients are rounded; books vary
slightly)
RGB -> HSV :   V = max(R,G,B);  S = (max-min)/max;  H = 60° x (position of the dominant colour):
               if max = R: H = 60 (G-B)/(max-min);   if max = G: H = 60 [2 + (B-R)/(max-min)];
               if max = B: H = 60 [4 + (R-G)/(max-min)];   add 360° if H is negative
```

**Example 8: Colour conversions**

**RGB → CMY:** (0.2, 0.6, 0.8) → C = 0.8, M = 0.4, Y = 0.2.
**RGB → YIQ (Y only):** pure red (1,0,0): Y = 0.299. For (0.2,0.6,0.8): Y = 0.0598 + 0.3522 + 0.0912 = **0.503**.
**RGB → HSV:** orange (1, 0.5, 0): max = 1 (R), min = 0. V = 1, S = (1−0)/1 = 1, H = 60 × (0.5 − 0)/1 = **30°**. So HSV = (30°, 1, 1).
**RGB → HSV:** (0.2, 0.6, 0.8): max = 0.8 (B), min = 0.2. V = 0.8, S = 0.6/0.8 = 0.75, H = 60[4 + (0.2−0.6)/0.6] = 60 × 3.333 = **200°**.

| Aspect | RGB | CMY(K) | YIQ | HSV / HLS |
|---|---|---|---|---|
| Mixing | Additive | Subtractive | Luma + chroma | Perceptual (hue, saturation, value/lightness) |
| Natural for | Light-emitting devices | Ink on paper | Broadcast / video | Humans choosing colours |
| Weak point | Hard to pick “a lighter orange” | Printer-dependent | Not intuitive for users | Not exactly the same on all screens |
| Black / White | (0,0,0) / (1,1,1) | (1,1,1) / (0,0,0) | Y = 0 / Y = 1 | V = 0 / S = 0, V = 1 |

## 11.3 Colour applications

| Application | Colour model / idea used | Why |
|---|---|---|
| Display on screen, cameras | RGB | Screens emit red, green, blue light |
| Printing (books, posters) | CMYK | Inks subtract light from white paper |
| TV broadcasting, video compression | YIQ (and related luma–chroma models such as YUV) | Colour detail can be reduced without much visible loss; old B/W TVs still work |
| Colour picker, photo editing | HSV / HLS | Easy to change brightness or saturation alone |
| Shading and lighting in 3-D | RGB | The illumination equation is applied separately to the R, G and B components |
| Data visualization (maps, heat maps) | Colour maps (sequential, diverging) | Colour encodes values; avoid relying on red–green alone for colour-blind users |
| Indexed-colour images and GIFs | Colour look-up table (palette) | Store a small index per pixel instead of a full colour |
| User interface and web design | RGB hex, HSL, contrast checking | Readability and accessibility |

**IMPORTANT**

Check your syllabus for the exact set of colour applications expected. The table above shows typical uses.

## 12 Common Student Mistakes

**Avoid these**

1. Writing the plane normal from vertices in the **wrong order**: the normal then points inward and the inside/outside test gets reversed.
2. Using a polygon with 4+ vertices as if it were always planar.
3. Saying a Bézier curve passes through **all** control points. It passes only through the **first and last**.
4. Stating that the degree of a Bézier curve is the number of control points (it is **points − 1**).
5. Thinking moving one Bézier control point changes only a part of the curve. It changes the **whole** curve; B-splines have local control.
6. Forgetting that blending functions sum to 1 and that the curve lies in the convex hull.
7. Mixing up the order d and the degree d − 1 of a B-spline, or miscounting the knots (n + d + 1).
8. Using u outside the valid parameter range, or mixing up u and v in surfaces.
9. Confusing **interpolation** (passes through points) with **approximation**.
10. Mixing up $C^1$ (equal tangent vectors) with $G^1$ (same direction only).
11. Using the wrong Hermite matrix, or forgetting that tangents are **vectors**, not points.
12. Confusing Phong **shading** with the Phong **reflection model**; saying Gouraud interpolates normals (it interpolates **intensities**).
13. Forgetting to **normalize** the interpolated normal in Phong shading.
14. Mixing additive (RGB) and subtractive (CMY) mixing rules, e.g. saying red + green = brown in light. In light, red + green = **yellow**.
15. Using the wrong formula for H in HSV (depends on which component is the maximum).

## 13 5-Minute Classroom Activity: “Curves, Shades and Colours”

**Task (work in pairs)**

1. **Bézier (2 min):** Control points p0 = (0,0), p1 = (0,4), p2 = (4,4), p3 = (4,0). Find P(0.5). (Use the weights 1/8, 3/8, 3/8, 1/8.)
2. **Colour (1.5 min):** Convert RGB (0.2, 0.6, 0.8) to CMY. Which colour is (1, 1, 0) in RGB?
3. **Shading (1 min):** Which shading method would you choose for (a) a cube in a quick preview, (b) a shiny sphere in a game cut-scene? Why?
4. **Share (30 s):** Compare answers with the next pair.

| Question | Answer |
|---|---|
| 1. Bézier P(0.5) | x = 0 + 0 + 3/8·4 + 4/8 = 1.5 + 0.5 = 2; y = 3/8·4 + 3/8·4 = 3. **P(0.5) = (2, 3)** |
| 2. Colour | CMY = (0.8, 0.4, 0.2). (1,1,0) = **yellow** (red + green light) |
| 3. Shading | (a) Flat: fast, faces are flat anyway. (b) Phong (per-pixel): smooth surface and a good specular highlight |

**Learning outcome**

Bézier points are weighted averages of control points; colour models are just different ways to describe the same colour; shading choice trades speed against quality.

## 14 Interview Questions

| # | Question | Key answer |
|---|---|---|
| 1 | Why are triangles preferred in polygon meshes? | Three points are always planar; hardware is optimised for triangles; any polygon can be split into triangles. |
| 2 | What do the vertex, edge and surface tables store? | Coordinates; vertex pairs for each edge; the edges (or vertices) of each polygon. |
| 3 | How do you find the normal of a polygon? | Cross product of two edge vectors (vertices in counter-clockwise order from outside). |
| 4 | What is a quadric surface? Give examples. | Second-degree equation surfaces: sphere, ellipsoid, paraboloid, hyperboloid. |
| 5 | Interpolation vs approximation spline? | Interpolation passes through control points; approximation only follows them. |
| 6 | What is a Hermite curve? | Cubic defined by two end points and two end tangents. |
| 7 | State four properties of Bézier curves. | Passes through end points; convex hull; global control; degree = points − 1; tangent at ends along polygon. |
| 8 | Why do we need B-splines when Bézier curves exist? | Local control and a degree independent of the number of control points. |
| 9 | What is a knot vector? | Non-decreasing list of n + d + 1 parameter values where curve pieces join. |
| 10 | $C^1$ vs $G^1$ continuity? | $C^1$: equal tangent vectors; $G^1$: same tangent direction only. |
| 11 | Gouraud vs Phong shading? | Gouraud interpolates intensities; Phong interpolates normals and evaluates lighting per pixel. |
| 12 | What are Mach bands? | Perceived bright/dark bands at intensity discontinuities, seen with flat shading. |
| 13 | Why does printing use CMYK and screens RGB? | Inks subtract light (subtractive); screens emit light (additive). |
| 14 | Why is YIQ useful for TV? | Y carries brightness (works for B/W sets); I and Q carry colour. |
| 15 | Where is HSV used? | Colour pickers and image editors, because hue, saturation and value are intuitive. |

## 15 University Examination Questions

### Short answer (2–3 marks)

1. What is a polygon mesh? Name the three tables used to store it.
2. Write the plane equation and explain the inside/outside test.
3. Write the parametric equations of a sphere.
4. Define a spline. Differentiate interpolation and approximation splines.
5. State the continuity conditions $C^0$, $C^1$, $C^2$.
6. Write the Hermite blending functions.
7. Write the equation of a Bézier curve and its blending functions.
8. List any four properties of Bézier curves.
9. What is a knot vector? How many knots are needed?
10. Compare flat and Gouraud shading.
11. Differentiate RGB and CMY colour models.
12. What is the purpose of the YIQ model?

### Medium answer (5–6 marks)

1. Find the plane equation through P1(2,0,0), P2(0,3,0), P3(0,0,6). *[Answer: $3x + 2y + z - 6 = 0$, normal (3,2,1)]*
2. Derive the Hermite basis matrix.
3. A Hermite curve has P0 = (0,0), P1 = (4,0), T0 = (0,4), T1 = (0,−4). Find P(0.5). *[Answer: (2, 1)]*
4. Compute the point at u = 0.5 on the cubic Bézier curve with control points (0,0), (1,2), (3,2), (4,0). *[Answer: (2, 1.5)]*
5. Explain how two Bézier curve segments are joined with $C^0$ and $C^1$ continuity.
6. Define B-spline basis functions (Cox–de Boor). Explain uniform, open uniform and non-uniform knot vectors.
7. Explain the Gouraud shading algorithm with the interpolation equations.
8. Explain Phong shading and compare it with Gouraud shading.
9. Convert RGB (1, 0.5, 0) to HSV. *[Answer: (30°, 1, 1)]*
10. Explain the HSV and HLS colour models with diagrams.

## 16 Five Long-Answer Questions

**DEFINITION**

1. Explain polygon surfaces: the vertex, edge and surface tables, consistency checks, the plane equation and the inside/outside test. Find the plane of the triangle with vertices (1,0,0), (0,1,0), (0,0,1) and test the points (0,0,0) and (1,1,1).

2. Describe spline representation. Explain interpolation and approximation splines and parametric and geometric continuity. Derive the Hermite curve with its matrix form and find P(0.5) for P0 = (0,0), P1 = (4,0), T0 = (0,4), T1 = (0,−4).

3. Explain the Bézier curve in detail: equation, blending functions, cubic matrix form, properties and how two segments are joined. Compute P(0.5) for the control points (0,0), (1,2), (3,2), (4,0).

4. Explain B-spline curves: the basis functions with the Cox–de Boor recursion, knot vectors (uniform, open uniform, non-uniform), properties and the comparison with Bézier curves. Explain Bézier and B-spline surfaces briefly.

5. Explain polygon rendering methods (constant, Gouraud and Phong shading) with equations and a comparison table. Then explain the RGB, CMY and HSV colour models with conversion formulas and give two colour applications.

**Answer key (numerical parts)**

- **Q1:** normal (1,1,1), plane $x + y + z - 1 = 0$; (0,0,0): −1 (inside); (1,1,1): +2 (outside).
- **Q2:** P(0.5) = (2, 1).
- **Q3:** P(0.5) = (2, 1.5).
- **Q4 (uniform cubic B-spline segment on the same four points):** P(0.5) = (2, 1.917).

# 17 Short Recap

**Remember**

- **Polygon mesh** = vertex + edge + surface tables. Plane: $Ax + By + Cz + D = 0$, normal = cross product of two edge vectors.
- **Quadrics** (sphere, ellipsoid, torus, ...) are exact equations; torus is usually listed with them.
- **Spline** = piecewise polynomial with continuity ($C^0$, $C^1$, $C^2$; $G^0$, $G^1$, $G^2$). Interpolation passes through points; approximation does not have to.
- **Hermite:** end points + tangents, $P(u) = [u^3\ u^2\ u\ 1]\cdot M_H \cdot G$.
- **Bézier:** $P(u) = \Sigma\ p_k\ C(n,k)\ u^k(1-u)^{n-k}$; ends interpolated; convex hull; global control; degree = points − 1.
- **B-spline:** order d, degree d − 1, n + d + 1 knots; local control; $C^{d-2}$ continuity; open knots give end-point interpolation; Bézier is a special case.
- **Surfaces:** tensor product of two curves; bicubic patch = 16 control points.
- **Shading:** flat (per polygon) → Gouraud (interpolate intensity) → Phong (interpolate normal, per-pixel lighting).
- **Colour:** RGB (additive, screens), CMY(K) (subtractive, print), YIQ (luma + chroma, TV), HSV/HLS (intuitive for users).